

Phishing Website Detection using Machine Learning

Final Project – Data Science in Cybersecurity

University: University of Haifa

Course: Data Science in Cybersecurity

Lecturer :Dr. Uri Itai

Date: 1.7.2026

Executive Summary:

This project evaluates a published machine learning solution for phishing website detection and examines whether the author's claims are supported by experimental results.

The original project uses handcrafted URL and website features to classify websites as phishing or legitimate. The dataset contains 10,000 websites, including 5,000 phishing websites and 5,000 legitimate websites. A total of 16 input features were used to train the machine learning models.

In this project, I implemented and evaluated six machine learning models: Logistic Regression, Decision Tree, Random Forest, Support Vector Machine (SVM), Multilayer Perceptron (MLP), and XGBoost. All models were trained using the same 80/20 train-test split and evaluated using Accuracy, Precision, Recall, F1-score, and the Confusion Matrix.

The results showed that XGBoost achieved the best overall performance, followed closely by Random Forest and Decision Tree. Support Vector Machine achieved the highest precision, while Multilayer Perceptron achieved a high recall. Logistic Regression produced the weakest overall performance.

The experiments also showed that the dataset is perfectly balanced, which simplifies the evaluation but does not fully represent real-world phishing prevalence.

Overall, the results support the author's main claim that machine learning is effective for phishing website detection. However, extending the comparison with additional models and discussing the dataset limitations provides a more complete and critical evaluation of the original project.

Summary of the Source:

➤ Problem Description :

Phishing is one of the most common cyberattacks used to steal personal information such as usernames, passwords, and financial data. Attackers create fake websites that look like real ones to trick users into giving their information.

As phishing attacks become more common and advanced, automatically detecting phishing websites has become an important task in cybersecurity.

➤ Why the Problem is Important? :

Phishing attacks can lead to financial loss, identity theft, and unauthorized access to personal or company accounts. Detecting phishing websites before users visit them helps reduce these risks and improve cybersecurity.

➤ The proposed solution:

The selected project uses machine learning to detect phishing websites. Instead of using manually created rules, it uses URL and website features to train models that can classify websites as either phishing or legitimate.

➤ The dataset used:

The dataset contains 10,000 websites, including 5,000 phishing websites from PhishTank and 5,000 legitimate websites from the University of New Brunswick dataset. For each website, 16 features were extracted from the URL, domain information, and website content.

➤ The model or methodology employed:

The original project extracted handcrafted features from each website and used them to train several machine learning models for phishing detection.

In this reproduction study, I implemented and evaluated six machine learning models: Logistic Regression, Decision Tree, Random Forest, Support Vector Machine (SVM), Multilayer Perceptron (MLP), and XGBoost.

All models were trained using the same 80/20 train-test split and evaluated using Accuracy, Precision, Recall, F1-score, and the Confusion Matrix to ensure a fair comparison.

Critical Evaluation:

➤ **The main claims:**

The original project claims that machine learning can effectively detect phishing websites using handcrafted URL and website features. It also suggests that tree-based models achieve the best performance for this task.

➤ **Are the Claims Supported by the Evidence? :**

The results obtained in this reproduction study generally support the author's main claims. All evaluated models achieved good classification performance, showing that the selected features contain useful information for distinguishing phishing websites from legitimate ones.

However, extending the evaluation with additional models provided new insights. Besides Logistic Regression, Decision Tree, and Random Forest, this project also evaluated Support Vector Machine (SVM), Multilayer Perceptron (MLP), and XGBoost. Among all evaluated models, XGBoost achieved the best overall performance, demonstrating that a broader comparison can lead to different conclusions.

➤ **Evaluation Methodology:**

The evaluation methodology is appropriate for this classification task. All models were trained using the same training and testing split and evaluated using Accuracy, Precision, Recall, F1-score, and the Confusion Matrix. Using multiple evaluation metrics provides a more complete assessment than relying on accuracy alone, especially for cybersecurity problems where different types of classification errors have different consequences.

➤ **Limitations:**

Although the results are encouraging, the project has several limitations.

First, the dataset contains only 10,000 websites and may not represent the wide variety of phishing techniques used in real-world attacks.

Second, the extracted features are manually designed and may not capture newly emerging phishing strategies.

Third, the dataset is perfectly balanced, containing 5,000 phishing and 5,000 legitimate websites. While this simplifies model training and evaluation, it does not reflect real-world phishing prevalence, where legitimate websites greatly outnumber phishing websites.

Finally, the evaluation is based on a single train-test split. Using cross-validation and additional datasets would provide a more reliable assessment of model performance.

➤ **Are the conclusions are justified?**

Overall, the experimental results support the author's main conclusion that machine learning is an effective approach for phishing website detection.

However, this reproduction study also shows that expanding the comparison with additional machine learning models changes the final ranking of the models. XGBoost achieved the best overall performance, while Random Forest and Decision Tree also performed well. This demonstrates that evaluating a broader set of models provides a more complete understanding of the strengths and limitations of the proposed approach.

Feature Engineering Analysis:

➤ **feature engineering performed:**

The original project extracted 17 features from each website. These features were taken from the URL, domain information, and the website content.

Instead of using the raw URL, each website was represented by a set of numerical features that were used to train the machine learning models.

➤ **Features Used:**

The extracted features are divided into three main groups:

- Address bar features, such as URL length, IP address, the '@' symbol, URL depth, URL redirection, TinyURL, and prefix or suffix in the domain.
- Domain features, such as DNS record, web traffic, domain age, and domain expiration time.
- HTML and JavaScript features, such as iframe usage, mouse-over behavior, disabled right-click, and website forwarding.

These features were chosen because they are common characteristics of phishing websites.

➤ **Data Transformations:**

Most of the features are binary, with values of 0 or 1, while URL depth is the only numerical feature.

All features were already represented in numerical form, so no categorical encoding was required. The dataset also contained no missing values, so no imputation was necessary.

Feature scaling was not applied because most of the evaluated models were tree-based and do not require normalized features. Dimensionality reduction was also not applied because the dataset contains only a small number of input features, making all of them easy to interpret.

➤ **The redundancy in the system:**

Some features may provide similar information. For example, URL length, URL depth, and TinyURL usage are all related to the URL structure.

Also, domain age and domain expiration time both describe the website domain and may contain similar information.

This can be checked using correlation analysis or feature importance. If some features are very similar, they can be removed without greatly affecting the model's performance.

Feature importance analysis and correlation analysis performed in this project showed that some features contribute much more than others. However, no features were removed because the number of features is relatively small and keeping them allows a fair comparison with the original project.

➤ **Was the Feature Engineering Meaningful:**

Overall, the feature engineering process was useful because the selected features are related to common phishing techniques.

Phishing websites often use long URLs, shortened links, suspicious domain names, or misleading HTML elements to trick users. The extracted features capture many of these characteristics.

By converting each website into numerical features, the machine learning models can learn these patterns and classify websites as phishing or legitimate.

➤ **Additional features that could potentially improve performance:**

The selected features provide useful information, but adding more features could improve the model.

For example, features such as the number of subdomains, suspicious words in the URL, SSL certificate validity, domain reputation, WHOIS information, website similarity, and URL analysis could be added.

These additional features could improve the ability of machine learning models to detect more sophisticated phishing attacks and may improve model generalization on newer datasets.

Reproducibility Analysis:

➤ **Code Execution:**

The original GitHub repository included the code and dataset needed to reproduce the experiments.

The code was successfully executed in Google Colab. All six machine learning models were trained and evaluated without major problems, and the experiments could be reproduced consistently.

➤ **Are all required files and dependencies available:**

The repository included the dataset and all the files needed to reproduce the experiments.

The implementation uses common Python libraries such as pandas, numpy, matplotlib, seaborn, scikit-learn, and XGBoost. These libraries are widely used, well documented, and easy to install.

A requirements.txt file is also provided to simplify the installation of the required Python libraries.

➤ **Hidden Preprocessing:**

No major hidden preprocessing steps were found.

The preprocessing process is clearly described. It includes loading the dataset, checking for missing values, selecting the input features, and splitting the data into training and testing sets.

Because these steps are simple and well documented, the experiments can be reproduced easily.

➤ **The overall reproducibility of the work:**

Overall, the project is reproducible. The dataset, notebook, source code, and implementation details are available, allowing other users to reproduce the experiments and compare the results.

Although the exact numerical results may vary slightly because of software versions or random initialization, the overall conclusions are expected to remain consistent.

Experimental Results:

➤ **Experimental Setup:**

The experiments were performed using the phishing website dataset, which contains 10,000 websites. The dataset was split into training and testing sets using an 80/20 ratio. Six machine learning models were implemented and evaluated:

- Logistic Regression
- Decision Tree
- Random Forest
- Support Vector Machine (SVM)
- Multilayer Perceptron (MLP)
- XGBoost

All models were trained on the same training data and evaluated on the same test set to ensure a fair comparison.

➤ **Modifications introduced:**

Compared with the original project, this reproduction study extends the evaluation by implementing three additional machine learning models: Support Vector Machine (SVM), Multilayer Perceptron (MLP), and XGBoost.

The project also includes a more detailed exploratory data analysis (EDA), feature importance analysis, feature grouping, class imbalance analysis, and a broader comparison of the trained models.

All models were evaluated using Accuracy, Precision, Recall, F1-score, and the Confusion Matrix.

➤ **The Models You Trained:**

Six machine learning models were trained and evaluated:

- Logistic Regression
- Decision Tree
- Random Forest
- Support Vector Machine (SVM)
- Multilayer Perceptron (MLP)
- XGBoost

All models were trained using the same training data and evaluated on the same testing set to ensure a fair comparison.

➤ **The evaluation metrics:**

The following evaluation metrics were used:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

These metrics give a better evaluation than accuracy alone because they measure both the correctness of the predictions and how well the models detect phishing websites.

➤ **The obtained results:**

XGBoost achieved the best overall performance with an accuracy of 87.05% and the highest F1-score.

Random Forest and Decision Tree also achieved strong performance and produced results very close to XGBoost.

Support Vector Machine achieved the highest precision but had a lower recall, meaning that it missed more phishing websites.

Multilayer Perceptron achieved competitive performance with one of the highest recall values.

Logistic Regression achieved the lowest overall performance among the evaluated models.

Overall, XGBoost provided the best balance between precision and recall, making it the strongest model in this study.

Error Analysis:

➤ Examples of Model Failures:

Although all six machine learning models achieved good performance, none of them classified every website correctly. Each model produced both False Positives and False Negatives.

The strongest model, XGBoost, achieved the highest overall accuracy (87.05%) and F1-score (86.19%), but it still made some classification errors, showing that phishing website detection remains a challenging classification problem.

➤ Patterns in the Errors:

The results show clear differences between the evaluated models.

Logistic Regression achieved the lowest recall (64.60%), meaning that it failed to detect many phishing websites.

Support Vector Machine achieved the highest precision (96.60%), but its recall (65.30%) was also relatively low. This means that although its phishing predictions were usually correct, it still missed many phishing websites.

Decision Tree, Random Forest, Multilayer Perceptron, and XGBoost achieved much higher recall values and detected a larger proportion of phishing websites.

Among all evaluated models, XGBoost achieved the best overall balance between precision and recall.

➤ Cybersecurity Implications of the Errors:

In phishing website detection, False Negatives are generally more dangerous than False Positives.

A False Negative allows a phishing website to be classified as legitimate, which may lead to stolen credentials, financial fraud, or identity theft.

A False Positive incorrectly classifies a legitimate website as phishing. Although this may inconvenience users, it is generally less harmful than allowing a phishing website to go undetected.

➤ The Trade-off Between False Positives and False Negatives:

Reducing False Negatives is especially important in cybersecurity because undetected phishing websites may cause significant damage.

Support Vector Machine focused on minimizing False Positives, which resulted in very high precision but lower recall.

In contrast, XGBoost, Random Forest, and Multilayer Perceptron achieved a better balance between precision and recall while still maintaining high overall performance.

Based on these results, XGBoost is the most suitable model for phishing website detection among the evaluated models.

Conclusions:

➤ **Key findings:**

In this project, I reproduced and extended a published machine learning solution for phishing website detection.

Six machine learning models were trained and evaluated: Logistic Regression, Decision Tree, Random Forest, Support Vector Machine (SVM), Multilayer Perceptron (MLP), and XGBoost.

Among all evaluated models, XGBoost achieved the best overall performance with the highest accuracy and F1-score, while Random Forest and Decision Tree also produced strong results.

➤ **Lessons learned:**

This project showed that evaluating machine learning models using multiple evaluation metrics provides a much better understanding of model performance than using accuracy alone.

The exploratory data analysis and feature importance analysis also showed that some features contribute more than others, helping explain why the best-performing models achieved strong performance.

➤ **Strengths and weaknesses of the proposed solution:**

One strength of the proposed solution is that it achieves good performance using a relatively small set of simple and interpretable handcrafted features.

However, the solution also has several limitations. The dataset is perfectly balanced and contains only one collection of phishing websites. In addition, handcrafted features may not capture newly emerging phishing techniques, which could reduce performance on real-world data.

➤ **Suggestions for future improvements:**

Future work could evaluate the models using cross-validation, larger and naturally imbalanced phishing datasets, and additional URL or website features.

Deep learning approaches and automated feature extraction methods could also be explored to improve phishing website detection.

Summing It Up:

This project reproduced and critically evaluated a published machine learning approach for phishing website detection.

The original project demonstrated that handcrafted URL and website features can be used to classify phishing websites effectively. In this reproduction study, the evaluation was extended by comparing six machine learning models instead of only the original baseline models.

The results support the author's main claim that machine learning is an effective solution for phishing website detection. However, the extended experiments showed that XGBoost achieved the best overall performance, while Random Forest and Decision Tree also performed very well.

The additional exploratory data analysis, feature importance analysis, class imbalance discussion, and broader and more systematic comparison provided a deeper understanding of both the dataset and the behavior of the evaluated models.

This study also highlights an important limitation of the original work. Although the balanced dataset simplifies model evaluation, it does not fully represent real-world phishing prevalence, where phishing websites are much less common than legitimate websites.

Overall, this reproduction study confirms the value of the original approach while providing a broader comparison of machine learning models and a more critical evaluation of the methodology, experimental design, and dataset limitations.